Software Requirements Specification

Jeans for Jeans — Cloud-Native E-Commerce Platform

Version 1.0

April 2026

Field	Details
Project	Jeans for Jeans E-Commerce Platform
Architecture	Microservices — 7 Spring Boot services
Frontend	React + Vite SPA
Messaging	Apache Kafka
Security	JWT — Spring Security
Cloud	AWS EC2 + S3

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions, Acronyms and Abbreviations 4
1.4 References 5
1.5 Document Overview 5
2. Overall Description 6
2.1 Product Perspective 6
2.2 Product Functions — Summary 6
2.3 User Classes and Characteristics 6
2.3.1 Customer (CUSTOMER role) 6
2.3.2 Administrator (ADMIN role) 6
2.3.3 External System — Stripe 7
2.3.4 System (automated processes) 7
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 7
2.6 Assumptions and Dependencies 7
3. Functional Requirements 9
3.1 Auth Service 9
3.1.1 User Registration (US-1, US-4) 9
3.1.2 Authentication (US-2) 9
3.1.3 Password Reset (US-3, US-30) 9
3.2 User Service 10
3.2.1 Profile Management (US-5) 10
3.2.2 Admin User Management (US-6) 10
3.3 Product Service 10
3.3.1 Product Catalog (US-8 through US-15) 10
3.3.2 Admin Product Management (US-12, US-28) 11
3.3.3 Internal Endpoints 11
3.4 Order Service 11
3.4.1 Shopping Cart (US-16, US-17) 11
3.4.2 Order Placement and Checkout (US-18 through US-21) 12
3.4.3 Order History and Status (US-22, US-23, US-24) 12
3.4.4 Admin Order Management 12

3.5 Payment Service 13
3.6 Notification Service 13
3.7 Review Service ..... **Error! Bookmark not defined.**
4. Non-Functional Requirements 14
4.1 Performance 14
4.2 Security 14
4.3 Reliability and Availability 15
4.4 Scalability 15
4.5 Maintainability 15
4.6 Portability and Deployment 16
5. System Architecture 17
5.1 Architecture Pattern 17
5.2 Technology Stack 17
5.3 Microservice Port Assignments 18
5.4 Kafka Topic Registry 18
6. External Interface Requirements 19
6.1 User Interface 19
6.2 API Gateway Interface 19
6.3 Stripe Payment Interface 20
6.4 AWS S3 Interface 20
6.5 Email Interface (SMTP) 21
7. Data Model Overview 22
7.1 Database Allocation 22
7.2 Cross-Service Data Contracts 22
7.3 Snapshot Strategy 22
8. System Constraints and Limitations 23
8.1 AWS Free Tier Constraints 23
8.2 Current Scope Exclusions 23
8.3 Known Technical Limitations 23
9. Requirements Traceability Matrix 25
10. Appendix 27
10.1 Order Status State Machine 27
10.2 Payment Status State Machine 27
10.3 Environment Variables Reference 27
10.4 Revision History ..... **Error! Bookmark not defined.**

1. Introduction

1.1 Purpose

Jeans for Jeans enables customers to browse products, place orders, and make payments through a web-based interface, while providing administrators with tools to manage the product catalog, monitor orders, and view business analytics. The system is designed for deployment on AWS infrastructure using Docker containers.

1.2 Scope

Jeans for Jeans is a full-stack web application composed of the following independently deployable microservices:

- Auth Service — user registration, authentication, JWT token management
- User Service — profile management, address book, customer support
- Product Service — product catalog, categories, variants, inventory, S3 image storage
- Order Service — shopping cart, checkout, order lifecycle management
- Payment Service — payment initiation, Stripe webhook processing, refunds
- Notification Service — event-driven email notifications via Kafka

The system is accessible via a React single-page application (SPA) that communicates with all services through a centralized Spring Cloud Gateway. All microservices are containerized with Docker and deployed on AWS EC2, with binary assets stored on AWS S3.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
SPA	Single Page Application — the React frontend
CRUD	Create, Read, Update, Delete — basic data operations
REST	Representational State Transfer — the API architectural style
Kafka	Apache Kafka — distributed message broker for async events
EC2	Amazon Elastic Compute Cloud — virtual server hosting
S3	Amazon Simple Storage Service — object storage for files
Docker	Container platform used to package and deploy each service
Gateway	Spring Cloud Gateway — single entry point for all API requests
BCrypt	Password hashing algorithm used by Spring Security
Microservice	Small, independently deployable service with a single responsibility

Term	Definition
DTO	Data Transfer Object — a class used to carry data between layers
Snapshot	A copy of data captured at a point in time, not a live reference

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- Spring Boot 3.2 Documentation — https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Cloud Gateway Documentation — https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/
- Apache Kafka Documentation — https://kafka.apache.org/documentation/
- AWS Free Tier Overview — https://aws.amazon.com/free/
- React 18 Documentation — https://react.dev/

1.5 Document Overview

Section 2 provides a general product description including context, user characteristics, constraints, and assumptions. Section 3 covers all functional requirements organized by microservice. Section 4 defines non-functional requirements including performance, security, and reliability. Section 5 describes system architecture and technology stack. Section 6 provides external interface requirements. Section 7 presents the data model overview. Section 8 defines system constraints and limitations.

2. Overall Description

2.1 Product Perspective

Jeans for Jeans is a system designed as a cloud-native microservices application. It does not replace or integrate with any existing system. The platform is self-contained and communicates with two external systems: Stripe for payment processing, and Gmail SMTP for outbound email delivery. All internal communication is either synchronous REST over the Docker network or asynchronous events over Apache Kafka.

The system is structured around a clear boundary: the API Gateway is the only entry point for external clients (the React SPA and Postman during testing). Internal service-to-service calls bypass the Gateway entirely and communicate using Docker container names as hostnames.

2.2 Product Functions — Summary

Feature area	Key capabilities	Service responsible
Authentication	Register, login, JWT, password reset, email verify	Auth Service
User profiles	Profile edit, photo upload, address management	User Service
Product catalog	Browse, search, filter, sort, variants, stock	Product Service
File storage	Product images and profile photos on S3	Product + User Service
Shopping cart	Add, update, remove items, cart count	Order Service
Checkout	Place order, shipping address, payment method	Order Service
Payments	Stripe integration, webhook, refunds	Payment Service
Order management	History, detail, cancellation, status tracking	Order Service
Notifications	Transactional emails for all key events	Notification Service
Admin dashboard	Stats, product CRUD, user management, orders	Multiple services

2.3 User Classes and Characteristics

2.3.1 Customer (CUSTOMER role)

Registered users who browse and purchase products. Customers can manage their own profiles, addresses, carts, orders, and reviews. They have read-only access to the product catalog and cannot access admin functionality. Customers interact via the React SPA exclusively.

2.3.2 Administrator (ADMIN role)

Platform operators who manage the product catalog, view all orders, manage users, and access the analytics dashboard. Administrators are created directly in the database during setup. All admin endpoints require the X-User-Role: ADMIN header, which is injected by the API Gateway after JWT validation.

2.3.3 External System — Stripe

Stripe is a payment gateway that calls the Payment Service webhook endpoint (POST /api/payments/webhook) to confirm or fail payment transactions. It is authenticated using HMAC signature verification, not JWT.

2.3.4 System (automated processes)

Kafka consumers that run within services and respond to events without user initiation. Examples: Order Service confirming an order after receiving a payment.completed event; Notification Service sending an email after receiving an order.placed event.

2.4 Operating Environment

- Server: AWS EC2 t2.micro (1 vCPU, 1 GB RAM), Amazon Linux 2023
- Runtime: Docker Engine 24+ with Docker Compose v2
- Database: MySQL 8 — single container with separate databases per service
- Message broker: Apache Kafka 3.x, both containerized
- Object storage: AWS S3 standard tier, single bucket

2.5 Design and Implementation Constraints

- All services must run within the AWS Free Tier. Each Spring Boot service is capped at -Xmx128m heap to stay within the 1 GB RAM limit of the t2.micro instance.
- The system uses a single MySQL container with multiple logical databases rather than multiple MySQL containers, to conserve memory.
- JWT access tokens expire in 15 minutes. Refresh tokens expire in 7 days.
- Product images are stored on AWS S3 only. The database stores the S3 URL, never binary data.
- All inter-service REST calls use Docker container names as hostnames. No service calls the API Gateway for internal communication.
- The /api/internal/* routes are never registered in the API Gateway routing table. They are accessible only within the Docker network.
- Stripe is used in test mode. No real payment cards are charged.

2.6 Assumptions and Dependencies

- A Stripe account with test API keys is available before Payment Service is deployed.
- An outbound SMTP account (e.g. Gmail App Password) is available for the Notification Service.

- An AWS account with S3 and EC2 access is available. An IAM user with scoped S3 permissions is created before deployment.
- Team members have Docker Desktop (local development) and SSH access to the EC2 instance (deployment).

3. Functional Requirements

3.1 Auth Service

3.1.1 User Registration

Requirement
The system shall allow a new user to register by providing a unique email address and a password of at least 8 characters.
Upon successful registration, the system shall store the password as a BCrypt hash. Plain-text passwords shall never be stored or logged.
Upon registration, the system shall create the user account with status UNVERIFIED and publish a user.registered Kafka event to trigger a verification email.
The system shall provide an email verification endpoint. When a user clicks the link in the verification email, their status shall change to ACTIVE.
The system shall reject login attempts from accounts with status UNVERIFIED or BLOCKED.

3.1.2 Authentication

Requirement
The system shall authenticate users via email and password and return a JWT access token (15-minute TTL) and a refresh token (7-day TTL) upon successful login.
The system shall provide a token refresh endpoint that accepts a valid refresh token and returns a new access token without requiring the user to re-enter credentials.
The system shall provide a logout endpoint that revokes the provided refresh token by deleting it from the database.
The JWT access token shall contain the user ID (sub claim) and role (role claim). No other sensitive data shall be encoded in the token payload.

3.1.3 Password Reset

Requirement
The system shall provide a forgot-password endpoint that accepts an email address and publishes a password.reset.requested Kafka event. It shall respond with 200 OK even if the email is not registered, to prevent user enumeration.
The Notification Service shall consume the password.reset.requested event and send an email containing a single-use reset token with a 1-hour expiry.
The system shall provide a reset-password endpoint that accepts a valid, unused, unexpired token and a new password, and updates the user's password hash accordingly.

3.2 User Service

3.2.1 Profile Management

Requirement
The system shall allow authenticated users to view their profile, including first name, last name, phone number, and profile photo URL.
The system shall allow authenticated users to update their first name, last name, and phone number.
The system shall allow authenticated users to upload a profile photo. The photo shall be stored on AWS S3 and the URL saved to the user profile.
The system shall allow authenticated users to add, view, update, and delete shipping addresses. Each address shall include recipient name, street, city, country, and zip code.

3.2.2 Admin User Management

Requirement
The system shall allow administrators to list all users with pagination, filterable by account status.
The system shall allow administrators to view the full details of any user account.
The system shall allow administrators to block a user account, preventing future login. Existing valid JWTs shall be rejected by the Auth Service on the next verification check.
The system shall allow administrators to permanently delete a user account and all associated profile data.

3.3 Product Service

3.3.1 Product Catalog

Requirement
The system shall provide a paginated product listing endpoint supporting filtering by category, price range, and brand, and sorting by price (asc/desc), rating, and creation date.
The system shall provide a keyword search endpoint that searches product name and description using a LIKE query.
The system shall provide a product detail endpoint returning name, description, base price, all images, all variants with stock status, and the average rating.
The system shall expose a recommended products endpoint returning the top 10 products ordered by total units sold descending.
Each product shall support multiple variants. A variant is defined by a combination of size and color attributes, and has its own price override, stock quantity, and SKU.

Requirement
When a variant's stock quantity reaches zero, its stock_status shall automatically be set to OUT_OF_STOCK. The system shall reject checkout requests for OUT_OF_STOCK variants.
The system shall support hierarchical categories (parent-child). A product belongs to exactly one category.

3.3.2 Admin Product Management

Requirement
Administrators shall be able to create a new product specifying category, name, description, base price, and initial status.
Administrators shall be able to upload multiple product images to AWS S3. The system shall store each image's S3 URL in the database. The first uploaded image shall be marked as the primary image.
Administrators shall be able to add, update, and delete product variants including size, color, price override, stock quantity, and SKU.
Administrators shall be able to create, update, and delete categories.

3.3.3 Internal Endpoints

Requirement
The Product Service shall expose an internal endpoint GET /api/internal/variants/{id} that returns variant details including stock status, price, and product name for use by the Order Service during cart item addition.
The Product Service shall expose an internal endpoint PUT /api/internal/variants/{id}/stock/decrement?qty={n} for the Order Service to call after a successful order placement to reduce variant stock.

3.4 Order Service

3.4.1 Shopping Cart

Requirement
Each authenticated user shall have exactly one persistent cart. The cart shall be created automatically on first item addition.
Adding a product variant that is already in the cart shall increment its quantity rather than creating a duplicate entry.
When an item is added to the cart, the system shall snapshot the current price and variant details from the Product Service. Subsequent price changes shall not affect the cart item.

Requirement
The cart shall support a maximum of 50 items. Attempts to add beyond this limit shall be rejected with HTTP 400.
The system shall provide a cart count endpoint returning the total quantity of all items for use in the frontend navigation badge.

3.4.2 Order Placement and Checkout

Requirement
The checkout endpoint shall verify the stock availability of all cart items by calling the Product Service before creating the order. If any item is out of stock, the request shall be rejected with HTTP 409.
A successful order placement shall: (1) persist the order with status PENDING, (2) decrement stock in the Product Service, (3) clear the user's cart, and (4) publish an order.placed Kafka event.
The order shall capture a shipping address snapshot at checkout time. This snapshot shall be stored with the order and shall not change if the user later edits their address book.
Order items shall store a snapshot of product name, variant label, image URL, unit price, quantity, and line total. These values shall not change if product details are later modified.

3.4.3 Order History and Status

Requirement
Authenticated users shall be able to retrieve a paginated list of their own orders, ordered by placement date descending.
Authenticated users shall be able to retrieve the full detail of any of their own orders, including all items, shipping address, and payment status fetched from the Payment Service.
Users shall be able to cancel an order only if its status is PENDING or CONFIRMED and the order was placed within the configured cancellation window (default 60 minutes). Cancellation shall publish an order.cancelled Kafka event.
The system shall automatically change an order's status from PENDING to CONFIRMED upon receiving a payment.completed Kafka event from the Payment Service.

3.4.4 Admin Order Management

Requirement
Administrators shall be able to list all orders with optional filtering by order status and pagination.

Requirement
Administrators shall be able to manually update the status of any order to any valid status (PROCESSING, SHIPPED, DELIVERED, CANCELLED).
The admin statistics endpoint shall return: total order count, orders placed today, total revenue, revenue today, pending order count, and cancelled order count.

3.5 Payment Service

Requirement
The payment initiation endpoint shall create a Stripe PaymentIntent and return the client_secret to the frontend for client-side payment confirmation.
The system shall expose a webhook endpoint (POST /api/payments/webhook) with no JWT requirement. It shall verify the Stripe HMAC signature before processing any event.
Upon receiving a successful payment event from Stripe, the system shall update the payment record to COMPLETED, record the paidAt timestamp, and publish a payment.completed Kafka event.
Upon receiving a failed payment event from Stripe, the system shall update the payment record to FAILED and publish a payment.failed Kafka event.
Upon receiving an order.cancelled Kafka event, the system shall initiate a Stripe refund for any completed payment associated with that order and publish a payment.refunded event.
Authenticated users shall be able to view their payment history and the payment status for a specific order.
Administrators shall be able to list all payment transactions and view details of any individual transaction.

3.6 Notification Service

Requirement
The service shall consume the user.registered Kafka event and send an email verification email containing a unique time-limited token link
The service shall consume the password.reset.requested Kafka event and send a password reset email containing a single-use token
The service shall consume the order.placed Kafka event and send an order confirmation email containing the order ID, items, total amount, and shipping address
The service shall consume the order.status.updated Kafka event and send a status update email to the customer when an order is marked as SHIPPED or DELIVERED.
The service shall consume the payment.completed Kafka event and send a payment confirmation email.
All email send attempts shall be logged to the notification_log table with status (SENT or FAILED), type, recipient, and a reference to the triggering entity.

Requirement
A failed email send shall be caught and logged. It shall not cause the Kafka consumer to crash or stop processing subsequent events.

4. Non-Functional Requirements

4.1 Performance

Requirement	Target
API response time for product listing and search	< 500 ms at 95th percentile
API response time for cart operations	< 300 ms at 95th percentile
Order placement (checkout) end-to-end	< 2 seconds including stock check
Product image upload to S3	< 3 seconds for images up to 5 MB
Order confirmation email delivery after order placed	< 30 seconds (Kafka async path)
All Docker containers reach healthy state on EC2	< 90 seconds from cold start

4.2 Security

Requirement
All API requests to protected endpoints must include a valid JWT access token. The API Gateway shall reject requests with missing, malformed, or expired tokens with HTTP 401.
Admin endpoints shall be accessible only to users with role ADMIN. The API Gateway shall return HTTP 403 for CUSTOMER role access to admin routes.
Passwords shall be hashed using BCrypt with a minimum cost factor of 10 before storage.
JWT access tokens shall expire after 15 minutes. Refresh tokens shall expire after 7 days.
The JWT secret key shall be loaded from an environment variable and shall never be committed to source control.
Stripe webhook requests shall be verified using HMAC-SHA256 signature verification before processing. Requests with invalid signatures shall be rejected with HTTP 400.
AWS S3 access keys and secrets shall be stored as environment variables, never hardcoded in source code.

Requirement
The /api/internal/* endpoints shall not be registered in the API Gateway. They are accessible only via internal Docker network communication.
CORS shall be configured on the API Gateway to allow only the frontend origin. All other origins shall be blocked.
All inter-service HTTP requests shall use container name resolution within the Docker network. No services shall be exposed directly to the public internet except via the Gateway (port 8080) and Nginx (port 80).

4.3 Reliability and Availability

Requirement
Each microservice shall recover automatically from crashes. All Docker Compose services shall be configured with restart: unless-stopped.
Kafka consumer errors shall be caught and logged. A single failed message shall not crash the consumer or block processing of subsequent messages.
Payment Service failures during stock decrement (after order placement) shall be non-fatal. The order is already saved; stock discrepancy shall be reconciled by an admin or scheduled process.
PaymentServiceClient calls from the Order Service shall be wrapped in try-catch. If the Payment Service is unavailable, the order detail endpoint shall return paymentStatus: "UNKNOWN" rather than failing the entire response.

4.4 Scalability

The system is designed for AWS Free Tier single-instance deployment. Horizontal scaling is not in scope for this project. However, the microservices architecture ensures that each service can be scaled independently if deployed to a larger infrastructure. The following design decisions support future scalability:

- Each service has its own database schema, enabling independent database scaling.
- Kafka decouples producers and consumers, enabling independent scaling of each.
- JWT-based authentication is stateless, requiring no shared session storage.
- S3 object storage scales independently of compute resources.

4.5 Maintainability

Requirement
All services shall follow the Controller → Service interface → ServiceImpl → Repository layered architecture.
All Kafka event DTOs shall be defined in a shared /common module. No service shall define its own duplicate event class.

Requirement
All configurable values (DB URL, service URLs, Kafka broker, JWT secret, S3 bucket) shall be externalized as environment variables and referenced in application.yml using Spring property binding.
Application logs shall use SLF4J with Logback. All log statements shall include a meaningful context (orderId, userId, etc.) for traceability.
All service entry points shall use Spring Boot's @SpringBootApplication annotation with no other custom bootstrapping.

4.6 Portability and Deployment

Requirement
Every microservice and the React frontend shall have a Dockerfile using multi-stage builds (build stage + runtime stage).
The Spring Boot runtime stage shall use eclipse-temurin:17-jre-alpine and set JAVA_OPTS=-Xmx128m to limit heap consumption on the t2.micro instance.
A single root-level docker-compose.yml shall define all containers, the shared Docker network, and health check dependencies using depends_on with service_healthy conditions.
The system shall run in a local development environment (Docker Desktop on Windows/Mac/Linux) and produce identical behavior to the AWS EC2 deployment.

5. System Architecture

5.1 Architecture Pattern

Jeans for Jeans uses a microservices architecture with the following structural characteristics:

- Each microservice is independently deployable and owns exactly one bounded domain.
- Services communicate synchronously via REST for user-facing request paths and asynchronously via Kafka for event-driven workflows.
- Every external request enters the system through the API Gateway, which is the only service exposed to the internet.
- Inter-service REST calls use Docker container names directly, bypassing the Gateway.
- Each service owns its database schema. Foreign keys across service databases are not used; shared data is referenced by ID only.

5.2 Technology Stack

Layer	Technology	Version	Purpose
Backend services	Spring Boot	3.2.4	Each microservice framework
API Gateway	Spring Cloud Gateway	2023.0.1	Routing, JWT filter
Security	Spring Security + JJWT	0.12.x	Authentication and authorization
ORM	Spring Data JPA + Hibernate	3.2.x	Database access layer
Messaging	Spring Kafka	3.1.x	Kafka producer/consumer
Service clients	RestTemplate	Built-in	Synchronous inter-service calls
Database	MySQL	8.0	Relational persistence per service
Frontend	React + Vite	18.x / 5.x	Single page application
Styling	Tailwind CSS	3.x	Utility-first CSS
HTTP client	Axios	1.x	API calls from React
Message broker	Apache Kafka	3.x	Async event streaming
Container	Docker + Compose	24.x / v2	Service packaging and orchestration
Compute	AWS EC2 t2.micro	Amazon Linux 2023	Cloud hosting (free tier)
Object storage	AWS S3	Standard	Images and binary assets
Payments	Stripe SDK	Java v24.x	Payment processing
Email	JavaMailSender + SMTP	Built-in	Transactional emails

5.3 Microservice Port Assignments

Service	Container name	Internal port	Description
API Gateway	api-gateway	8080	Public entry point — mapped to host port 8080
Auth Service	auth-service	8081	Registration, login, JWT
User Service	user-service	8082	Profiles, addresses
Product Service	product-service	8083	Catalog, variants, S3 images
Order Service	order-service	8084	Cart, checkout, orders
Payment Service	payment-service	8085	Stripe, webhooks, refunds
Notification Service	notification-service	8086	Kafka consumers, email
Review Service	review-service	8087	Ratings and reviews
(frontend)	frontend	80	React SPA — mapped to host port 80
MySQL	mysql	3306	Single container, 4 databases
Kafka	kafka	9092	Message broker

5.4 Kafka Topic Registry

Topic name	Producer	Consumer(s)	Trigger
user.registered	Auth Service	Notification Service	New user registration
user.password-reset-requested	Auth Service	Notification Service	Forgot password request
order.placed	Order Service	Notification Service, Product Service	Successful checkout
order.cancelled	Order Service	Payment Service, Notification Service	User cancels order
order.status.updated	Order Service	Notification Service	Admin updates status
payment.completed	Payment Service	Order Service, Notification Service	Stripe payment success
payment.failed	Payment Service	Notification Service	Stripe payment failure

6. External Interface Requirements

6.1 User Interface

The user interface is a React 18 single-page application (SPA) built with Vite and styled using Tailwind CSS. It communicates exclusively with the API Gateway using Axios. The frontend is served by Nginx on port 80 and Nginx proxies /api/* requests to the Gateway on port 8080.

The SPA must support the following pages and views:

Page/View	Route	Access
Home / product listing	/	Public
Product detail	/products/:id	Public
Search results	/products?q=keyword	Public
Register	/register	Public (redirect if logged in)
Login	/login	Public (redirect if logged in)
Email verification	/verify-email?token=...	Public
Shopping cart	/cart	Authenticated
Checkout	/checkout	Authenticated
Order history	/orders	Authenticated
Order detail	/orders/:id	Authenticated
My profile	/profile	Authenticated
Admin dashboard	/admin	ADMIN role
Admin — products	/admin/products	ADMIN role
Admin — orders	/admin/orders	ADMIN role
Admin — users	/admin/users	ADMIN role

6.2 API Gateway Interface

All client requests are routed through the API Gateway. The Gateway applies a JWT validation filter to all routes except those on the public whitelist. The following table defines the routing rules:

Path prefix	Routed to	JWT required	Role required
/api/auth/**	auth-service:8081	No (public whitelist)	None
/api/users/**	user-service:8082	Yes	CUSTOMER or ADMIN
/api/admin/users/**	user-service:8082	Yes	ADMIN only

Path prefix	Routed to	JWT required	Role required
/api/products (GET)	product-service:8083	No	None
/api/products (POST/PUT/DELETE)	product-service:8083	Yes	ADMIN only
/api/admin/products/**	product-service:8083	Yes	ADMIN only
/api/cart/**	order-service:8084	Yes	CUSTOMER or ADMIN
/api/orders/**	order-service:8084	Yes	CUSTOMER or ADMIN
/api/admin/orders/**	order-service:8084	Yes	ADMIN only
/api/payments/**	payment-service:8085	Yes (except webhook)	CUSTOMER or ADMIN
/api/payments/webhook	payment-service:8085	No (Stripe HMAC)	None
/api/reviews/**	review-service:8087	Partial (reads public)	Writes: CUSTOMER

The Gateway injects two headers into every forwarded request after successful JWT validation: X-User-Id (the user UUID from the JWT sub claim) and X-User-Role (the role from the JWT role claim). Downstream services read these headers directly and do not re-parse the JWT.

6.3 Stripe Payment Interface

The Payment Service integrates with the Stripe API using the official Stripe Java SDK. The integration has two flows:

- Outbound — Payment Service calls Stripe's PaymentIntents API to create a payment session and returns the client_secret to the frontend. The frontend uses Stripe.js to complete the payment on the client side.
- Inbound — Stripe calls the Payment Service webhook endpoint with signed event payloads. The service verifies the HMAC-SHA256 signature using the Stripe-Signature header before processing.

6.4 AWS S3 Interface

Product Service and User Service upload binary assets to AWS S3 using the AWS SDK v2 for Java. The integration follows this pattern:

- Receive MultipartFile from the HTTP request.
- Generate a unique S3 key (e.g. products/{productId}/{uuid}.jpg).
- Upload using PutObjectRequest to the configured S3 bucket.
- Build and return the public URL (https://{bucket}.s3.amazonaws.com/{key}).
- Store the URL in the database. Binary data is never stored in the database.

6.5 Email Interface (SMTP)

The Notification Service sends transactional emails using Spring Boot's JavaMailSender configured with Gmail SMTP (smtp.gmail.com, port 587, STARTTLS). All emails use HTML templates rendered by a template engine (Thymeleaf or plain StringBuilder). Credentials are loaded from environment variables MAIL_USERNAME and MAIL_PASSWORD.

7. Data Model Overview

7.1 Database Allocation

Database	Service owner	Key tables
auth_db	Auth Service	users, refresh_tokens, email_tokens
user_db	User Service	user_profiles, addresses, support_tickets
product_db	Product Service	categories, products, product_images, product_variants
order_db	Order Service	carts, cart_items, orders, order_items, order_addresses
payment_db	Payment Service	payments, refunds
notification_db	Notification Service	notification_log

7.2 Cross-Service Data Contracts

Because each service owns its own database, cross-service data sharing is achieved through two mechanisms:

- Shared IDs — Every service uses the same user UUID (originally created by Auth Service) as the primary user identifier. A user record in user_db references the same ID as a record in auth_db. No foreign key constraint exists across databases; the ID is simply trusted.
- Kafka event payloads — Events carry all necessary data for consumers. For example, the OrderPlacedEvent contains the full shipping address and all item details so that Notification Service can send a confirmation email without calling back to the Order Service.

7.3 Snapshot Strategy

Order items store a full snapshot of product data at checkout time (product name, variant label, image URL, unit price). This is a deliberate design choice that ensures:

- Order history is immutable — a price change or product deletion does not retroactively alter past orders.
- The Order Service has no runtime dependency on the Product Service for reading order history — all data is self-contained.
- Cart items also snapshot price and product details at add-time. The checkout price is the price at order placement, not at add-to-cart time.

8. System Constraints and Limitations

8.1 AWS Free Tier Constraints

- The EC2 t2.micro instance provides 1 vCPU and 1 GB RAM. All containers combined must stay within this limit. Each Spring Boot service is capped at 128 MB heap using -Xmx128m.
- AWS Free Tier provides 750 hours of EC2 t2.micro usage per month. Running one instance continuously consumes approximately 744 hours per month, leaving 6 hours buffer.
- AWS S3 Free Tier provides 5 GB of standard storage and 20,000 GET requests per month. Product images and profile photos must stay within this budget for the project duration.
- RDS Free Tier (optional) provides one db.t3.micro instance for 12 months. If used, it replaces the MySQL Docker container but adds latency to startup.

8.2 Current Scope Exclusions

The following features are explicitly out of scope for the initial MVP and are deferred to stretch goals:

Feature	Reason excluded	Future consideration
Google OAuth2 login	Adds OAuth2 client config complexity	Spring Security OAuth2 client
Real-time order tracking	Requires WebSocket or SSE infrastructure	Spring WebFlux + SSE
Multi-seller marketplace	Changes entire product ownership model	Requires seller entity and role
Elasticsearch search	Exceeds EC2 RAM budget	Upgrade to t3.small first
Redis for cart storage	Extra container, EC2 RAM budget	Replace when scaling horizontally
HTTPS / SSL	Requires domain name + Let's Encrypt	Add Nginx certbot container
CI/CD pipeline	GitHub Actions or AWS CodePipeline	After MVP is stable
Product recommendations (ML)	No ML infra on free tier	SQL best-seller ranking only

8.3 Known Technical Limitations

- Kafka and Zookeeper together consume approximately 400 MB RAM. On a t2.micro, this leaves approximately 600 MB for all Spring Boot services and MySQL, which requires careful heap tuning.
- The system does not implement distributed transactions. The order placement flow (save order → decrement stock → clear cart → publish event) is not atomic across services. A

stock decrement failure after order save results in a stock discrepancy that must be corrected manually.

- Kafka messages are not encrypted in transit within the Docker network. For a production deployment, Kafka TLS and SASL authentication should be configured.
- The current rate limiting implementation uses an in-memory counter. On a multi-instance deployment this would not work correctly. Redis-based rate limiting should be used for scaling.
- Product search is implemented using SQL LIKE queries, which do not support full-text relevance ranking, stemming, or fuzzy matching. For production, Elasticsearch or MySQL FULLTEXT indexes should be used.

9. Requirements Traceability Matrix

The following table maps the 30 original user stories to functional requirements defined in Section 3.

Description (abbreviated)
Register with email and password
Login with email/password
Reset password via email
Verify email after registration
Edit profile (name, phone, photo, addresses)
Admin manage users (view, block, delete)
Contact customer support
View products by category
Search products by name/keyword
Filter and sort products
View product details
Admin add/edit/delete products
Choose product variants (size, color)
Out of stock display when stock = 0
See recommended products
Add products to cart
View and modify cart
Proceed to checkout from cart
Enter/choose shipping address at checkout
Choose payment method
Place order and receive confirmation
View order history and track status
Cancel an order
Auto-update status after payment
Rate and review product after receipt
See all reviews and average rating
Admin dashboard statistics
Admin manage categories and products

Description (abbreviated)
Email notifications for order confirmation and status
Password reset email

10. Appendix

10.1 Order Status State Machine

From status	To status	Trigger	Actor
(new)	PENDING	Order placed by customer	Customer (checkout)
PENDING	CONFIRMED	payment.completed Kafka event received	System (Kafka consumer)
PENDING	CANCELLED	Customer cancels within window	Customer
CONFIRMED	CANCELLED	Customer cancels within window	Customer
CONFIRMED	PROCESSING	Admin marks order as being prepared	Administrator
PROCESSING	SHIPPED	Admin marks order as shipped	Administrator
SHIPPED	DELIVERED	Admin marks order as delivered	Administrator
CANCELLED	REFUNDED	Stripe refund completed	System (Payment Service)

10.2 Payment Status State Machine

From status	To status	Trigger
(new)	PENDING	Payment initiated by customer
PENDING	COMPLETED	Stripe webhook — payment_intent.succeeded
PENDING	FAILED	Stripe webhook — payment_intent.payment_failed
COMPLETED	REFUNDED	Refund initiated after order cancellation

10.3 Environment Variables Reference

Variable	Service(s)	Description	Example
DB_HOST	All services	MySQL container hostname	mysql
DB_NAME	Each service	Database name for this service	order_db
DB_USER	All services	MySQL username	root
DB_PASS	All services	MySQL password	(secret)
KAFKA_SERVERS	All services	Kafka bootstrap server	kafka:9092
JWT_SECRET	Auth + Gateway	HMAC secret for JWT signing	(min 32 chars)
JWT_ACCESS_TTL_MS	Auth Service	Access token TTL in ms	900000

Variable	Service(s)	Description	Example
JWT_REFRESH_TTL_MS	Auth Service	Refresh token TTL in ms	604800000
PRODUCT_SERVICE_URL	Order Service	Product Service base URL	http://product-service:8083
PAYMENT_SERVICE_URL	Order Service	Payment Service base URL	http://payment-service:8085
ORDER_SERVICE_URL	Review + Payment	Order Service base URL	http://order-service:8084
STRIPE_API_KEY	Payment Service	Stripe secret key (test mode)	sk_test_...
STRIPE_WEBHOOK_SECRET	Payment Service	Stripe webhook signing secret	whsec_...
AWS_ACCESS_KEY_ID	Product + User	AWS IAM access key	(from IAM user)
AWS_SECRET_ACCESS_KEY	Product + User	AWS IAM secret key	(from IAM user)
AWS_S3_BUCKET	Product + User	S3 bucket name	Jeans for Jeans-assets
AWS_REGION	Product + User	AWS region	us-east-1
MAIL_USERNAME	Notification	SMTP username / sender email	noreply@Jeans for Jeans.com
MAIL_PASSWORD	Notification	SMTP app password	(Gmail App Password)
VITE_API_URL	React frontend	API Gateway base URL	http://{EC2-IP}:8080